

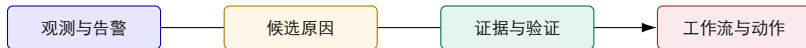
面向 AI 原生系统的智能运维

第 9 讲 | 智能体融入运维 workflow

从问答、查询和巡检，到诊断、SOP 与受控执行

陈鹏飞 | 中山大学

从第 8 讲继续：诊断结论如何进入 workflows?



第 7 讲 从观测数据构造故障上下文。

第 8 讲 用因果假设和干预验证。

本讲 让智能体把结论变成可审计的运维步骤。

审批失效算例：同意“重启”究竟批准了什么？

构造一个受控重启请求。审批绑定动作摘要；执行前必须重新比较目标、参数和版本。

检查项	审批时	执行时	决策
目标	cluster-A / Pod UID=u1	仍为 u1	通过目标核对
动作参数	重启 1 个副本	变为重启全部副本	拒绝；重新审批
配置版本	revision=7	已变为 revision=8	重新规划并评估
有效期	10:00–10:05	10:08 才开始	审批已过期

动作摘要可绑定：目标 UID、动作类型、参数、配置版本、审批范围和有效期。

推导与判断 审批的是一份具体动作计划；只保存“用户说了可以”无法防止参数扩大或目标漂移。摘要校验之外仍需执行端权限检查。

算例使用假设数据。

开场案例：凌晨两点，告警已经不是问题本身

值班群里出现

- ▶ frontend-proxy: 200 → 503;
- ▶ backend readiness: 仍然正常;
- ▶ 过去 10 分钟出现 3 类告警;
- ▶ 值班工程师不知道先看网络、端点还是负载。

一个“会聊天”的系统会做什么？

- ▶ 复述告警：“服务不可用，请检查网络。”
- ▶ 一个运维 Agent 应该：提出假设、取证、说明不确定性、给出最小动作；
- ▶ 若要执行动作，还要说明风险、审批人和回滚条件。

诊断问题 “能回答” 与 “能参与运维执行” 之间，缺的不是一句更长的提示词，而是一套 workflow。

本讲边界： workflow 进入运维，不是泛讲 Agent

本讲覆盖

- ▶ 问答、只读查询、日常巡检；
- ▶ 告警归并、故障排查、RCA；
- ▶ SOP、工单、审批、受控执行；
- ▶ 轨迹、证据、回放和持续优化。

留给第 10 讲

- ▶ 企业组织与岗位重构；
- ▶ 数据治理、知识产权与平台化；
- ▶ 从试点到规模化的转型路线；
- ▶ 预算、采购与长期运营模型。

非目标：让一个开放式 Agent 无审批地执行所有生产变更。

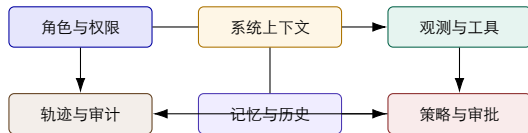
2 学时路线图

1. 数字分身与自治阶梯：从“回答”到“行动”；
2. workflow 机制：ReAct、状态机、Tape、工具接口约定；
3. 运维任务：查询、巡检、告警、RCA、SOP、工单；
4. 近期研究：多模态协作、告警生命周期、跨层追踪；
5. AgenticSRE 与项目探索：技能、工作台、HITL、ActionStack；
6. 三个综合案例：网络分区、KV Cache、审批与复盘。

一、数字分身与自治阶梯

先把“智能体”放进真实的运维任务里

“运维数字分身” 到底是什么？



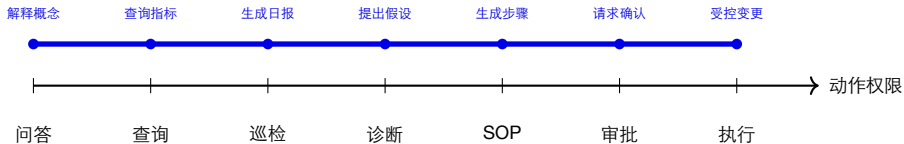
核心定义 这里的数字分身是服务于特定运维岗位的助手：知道负责哪些系统、能查哪些数据、能执行哪些动作。

四种形态：同一句话，责任完全不同

形态	主要输入	主要输出	是否改变系统
Chatbot	自然语言问题	一段回答或链接	否
Copilot	问题 + 当前上下文	建议、查询、草稿、摘要	否，通常由人执行
Agent	目标 + 工具 + 环境	多步轨迹、证据、结果	可以，但需要边界与策略
运维数字分身	角色 + 服务范围 + 长期知识 + 流程	面向某类值班/研发角色的持续协作	分级允许，必须可审计

不是 “Agent 一定比 Copilot 高级”，而是责任、状态和动作边界发生了变化。

把运维任务放到能力谱上



执行边界 每向右走一步，新增的不只是工具调用，还包括状态、风险、证据和责任。

自治阶梯：L0 到 L5，不是一次性跳跃

级别	智能体做什么	人做什么	适合的运维任务
L0	不参与	全部处理	传统值班
L1	解释与检索	判断是否可信	文档问答、命令说明
L2	只读查询与摘要	选择后续路径	健康检查、日报
L3	多步诊断与建议	复核证据与风险	告警归并、RCA 草稿
L4	低风险动作自动执行	处理异常与高风险动作	限流、重试、隔离
L5	在授权范围内执行并检查结果	设定边界、抽查、接管	已验证的 SOP 子流程

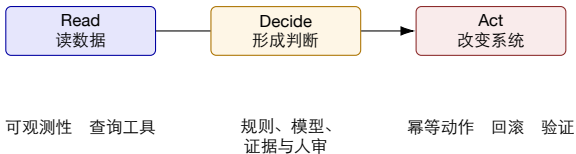
L5 只描述 “某个受约束任务域”，不是整个企业系统都达到 L5。

风险 × 自治：权限应该跟动作走

动作风险	典型例子	默认自治	必须记录
低 中 高	查询指标、读日志、生成摘要 重试请求、切换只读副本、限流 删除资源、扩容、回滚版本、改 网络策略	可自动 建议 + 单人确认 双人/值班负责人审批	查询范围、时间窗、来源 影响面、超时、回滚 变更单、审批链、回归验证

关键区分 诊断置信度高，不代表动作风险低；动作风险高，也不代表诊断必须完全交给给人。

把 read、decide、act 三层拆开



工程上最危险的设计，是让同一个自由文本输出同时承担判断和执行。

角色不是 **Prompt** 里的称呼，而是权限和信息范围

值班工程师 看到当前告警、SLO、变更与Runbook；可以批准中风险动作。

服务 **Owner** 看到版本、代码变更、依赖和历史事故；可以确认业务影响。

平台管理员 看到集群、网络、容量和策略；承担高风险变更责任。

- ▶ 同一个“根因”问题，三个角色可见证据不同；
- ▶ 角色边界决定检索范围、工具白名单和审批人；
- ▶ 数字分身的个性化首先是工作上下文个性化，而不是语气个性化。

一个运维助手需要哪些部件？

1. 角色与服务范围；
2. 当前上下文与时间窗；
3. 查询、诊断、变更工具；
4. SOP、策略和审批规则。
5. 短期状态与长期知识；
6. 证据、来源和不确定性；
7. 动作、回滚与终态验证；
8. 轨迹、审计和评测标签。

设计检查 删掉其中任何一项，系统都可能变成“能说、但不能复核”的 Demo。

生成文本不是系统事实：建立证据账本

字段	示例	作用
来源	Prometheus 查询、LogQL、TraceID、变更单	让人能复查原始证据
时间	2026-08-21 01:58-02:05	防止跨事故混用上下文
变换	聚合、过滤、采样、脱敏	说明模型看到的不是原始全集
结论	“NetworkChaos 是高概率假设”	明确它仍是判断，不是事实
验证	删除对象后 90 秒内 503 恢复	把干预与结果绑定

证据账本可以由结构化事件、Tape 或 Trace 保存；自然语言报告只是它的一个视图。

案例 A：每日巡检数字分身——输入

每天 09:00 自动触发

- ▶ 服务清单、SLO 与负责人；
- ▶ 过去 24 小时指标、日志、Trace 摘要；
- ▶ 发布、配置、扩缩容、网络策略变更；
- ▶ 过去 7 天相似事故与处理结果。

输出格式约定

- ▶ 正常项：一句话 + 查询链接；
- ▶ 异常项：症状、证据、影响面、候选原因；
- ▶ 建议动作：只读、建议、需审批三类；
- ▶ 未知项：明确“目前无法判断”的原因。

案例 A：巡检报告不是日报作文

服务	结论	可复核证据
checkout retriever payment	黄色：p95 延迟升高，但错误率正常 红色：向量库回源增加，可能导致 TTFT 上升 绿色：SLO 正常	指标查询、部署版本、流量对照 cache hit、query length、最近索引发布 SLO 面板、最近告警为空

报告结构 报告把“事实、推测、建议”分栏，值班人员可以先看影响最大的红项，再决定是否接管。

第一部分小结：数字分身的最小承诺

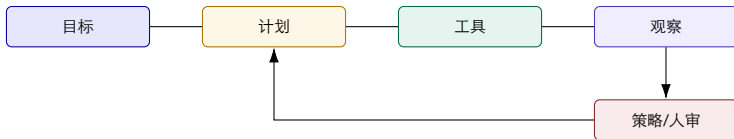
1. 它知道自己服务谁、能看到什么、不能做什么；
2. 它把输出拆成事实、判断、建议和动作；
3. 它按任务风险选择自治等级；
4. 它用证据和轨迹让人可以复核、接管、回滚。

过渡 有了“做什么”的边界，下一步要解决“怎么一步步做且不会失控”。

二、智能体 workflow 机制

从自由文本循环，到有状态、可恢复、可审计的执行

增强型 LLM：模型只是循环中的一个部件



生产系统还需要：超时、重试、幂等、权限、缓存、脱敏、回滚和终态检查。

ReAct: Reason + Act + Observation

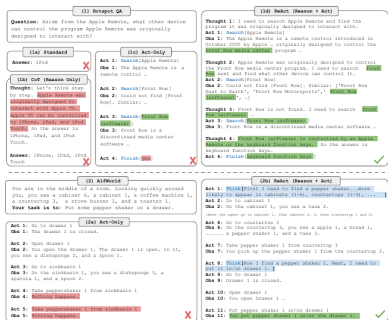


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotpotQA (Yang et al. [2018] question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AIWorld (Shridhar et al. [2020]) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

answers from questions in arithmetic, commonsense, and symbolic reasoning tasks (Wei et al. [2022]). However, this “chain-of-thought” reasoning is a static black box, in that the model uses

- Thought: 当前对任务的中间判断；
- Action: 选择一个外部动作；
- Observation: 环境返回的结果；
- 再根据观察修正下一步。

运维映射 把“看指标—查日志—比对变更—形成假设—验证”变成有证据的循环。

ReAct 的工程风险：循环并不自动等于可靠

无信息搜索 工具返回空、重复或模糊结果，模型仍继续“想”。

错误传播 早期假设被写入后续上下文，越查越像真的。

动作越权 文本中的建议被误当成可以直接执行的命令。

- ▶ 为每次工具调用定义输入/输出 schema 和失败语义；
- ▶ 为循环定义最大步数、信息增益阈值与人工接管条件；
- ▶ 让“证据不足”成为合法终态，而不是继续编故事。

Plan-and-Execute：先列步骤，再逐步完成



- ▶ 优点：步骤可展示、可计数、可插入审批；
- ▶ 风险：计划可能过时，执行结果可能改变后续条件；
- ▶ 运维折中：把稳定部分固化为 SOP，把异常分支交给模型重新规划。

StateFlow: 把 workflow 显式建模为状态机

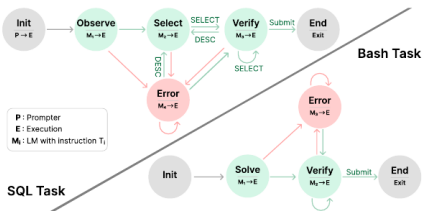


Figure 1: The StateFlow models for the SQL and Bash task. Init and End state are basic components of state machines, and states like Observe, Solve, Verify, Error can be adapted across various tasks. When reaching a state, a sequence of output functions defined by the model is executed (e.g., $M_i \rightarrow E$ means to first call the model and then call the SQL/Bash execution). Execution outcomes are indicated by red arrows for failures and green for successes. Transition to different states is based on specific rules. For example, at a success 'Submit'

- ▶ Init: 装载目标和上下文；
- ▶ Observe: 读取外部事实；
- ▶ Solve: 提出或执行局部步骤；
- ▶ Verify: 检查结果与不变量；
- ▶ Error / End: 失败接管或正常终止。

每个状态都可以是工具、提示词、规则或程序函数；状态转移由结果决定。

状态机带来三个“可问责”的问题

1. 当前状态是什么？——不是“模型大概在想什么”，而是结构化字段；
2. 什么条件允许转移？——成功、失败、超时、证据不足都要写清楚；
3. 终态是什么？——提交成功、回滚完成、人工接管和无法判断都不同。

state quad guard quad transition quad terminal

状态、守护条件、不变量：把隐含规则写出来

状态	允许动作	必须保持的不变量
Observe Solve Verify Approve Act	读指标、日志、Trace、变更 生成查询、比较候选、调用只读工具 回放、灰度、健康检查 提交变更请求 执行幂等动作	不写生产；查询时间窗明确 每个候选必须绑定证据来源 不扩大影响面；验证窗口结束才算完成 审批人、风险、回滚步骤齐全 权限白名单、超时、并发和回滚有效

设计原则 如果某条规则不能被状态机或策略检查器表达，就不要把它只藏在 Prompt 里。

错误、超时与补偿：在流程中明确处理失败

Error 工具失败、证据冲突或权限不足；转人工或重规划。

Timeout 在预设窗口内没有新信息；停止循环，保留轨迹。

Compensation 动作已部分完成；执行逆操作或标记需要人工修复。

- ▶ 重试不是“再问一次模型”；要区分可重试错误与永久错误；
- ▶ 补偿动作应提前写入 Runbook，而不是事故发生后现场生成；
- ▶ 每一种异常都要能回到明确的终态。

SOP：写清步骤、执行条件与失败处理



- ▶ 前置条件：目标对象、版本、权限、维护窗口；
- ▶ 步骤：输入 schema、幂等键、超时与输出 schema；
- ▶ 检查点：健康指标和业务验证；
- ▶ 回滚：何时停、怎么逆向、谁接管。

SOP 也要允许异常分支

步骤	正常路径	异常路径
确认影响面 采集证据 验证假设 执行动作	只读查询返回单服务 指标与日志都有数据 单次灰度通过 幂等调用成功	依赖拓扑冲突，扩大到上游/下游 数据源延迟或缺失，标记未知 结果不显著，增加对照或人工复核 部分成功，转补偿或人工接管

好的 SOP 不是把所有情况写完，而是把不可接受的情况显式地导向安全终态。

TapeAgents: 把思考、动作和观察写成可回放 Tape

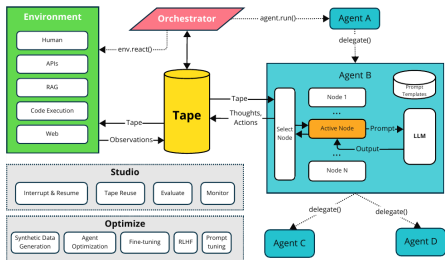


Figure 1: TapeAgents at a glance. The orchestrator alternates between running the agent and the environment interacting with each other via adding steps to the tape: a comprehensive, replayable semantic-level log. Agents are composed of basic reasoning units that we call nodes. The agents are organized in hierarchical teams, with one agent being active at a time. The tape and the agent configurations are highly structured and linked with rich metadata that supports the implementation of broadly usable developer tools (collectively called Studio) and optimization methods (collectively called Optimize).

- ▶ Tape 是结构化语义日志，也是可恢复状态；
- ▶ Agent、环境和编排器围绕 Tape 交替推进；
- ▶ Studio 支持中断、恢复、复用、评测和监控；
- ▶ Optimize 可以基于历史轨迹做提示词、代理或数据优化。

Tape 的最小事件结构

事件类型	关键字段	运维用途
Thought / Plan	目标、假设、当前状态	解释下一步为什么出现
Action	工具名、参数、权限、幂等键	重放、审计和权限检查
Observation	返回摘要、来源、时间窗、错误	形成可复核证据
Decision	选择、置信度、风险、审批人	区分模型判断与人决定
Terminal	成功、失败、接管、回滚	定义任务是否真正结束

注意 不要把全部原始日志直接塞进 Tape；需要分层、脱敏和可追溯引用。

可恢复性：事故中断后，不必从 **Prompt** 重新开始

1. 记录最后一个已确认的状态和证据水位；
2. 恢复时重新检查外部环境是否变化；
3. 只重放可安全重放的读操作；
4. 对写操作要求幂等键或人工确认；
5. 将“原轨迹”和“恢复后的分支”并列保存。

恢复不是盲目重放：外部系统已经变化时，旧观察可能不再成立。

ACI: 模型能不能用好工具，取决于接口设计

差的工具接口

- ▶ 一个命令接收任意字符串；
- ▶ 返回一大段未结构化文本；
- ▶ 失败与空结果混在一起；
- ▶ 没有权限、超时和副作用提示。

好的 Agent-Computer Interface

- ▶ 参数和返回值有 schema；
- ▶ 查询与变更明确分离；
- ▶ 错误可分类、可重试；
- ▶ 输出包含来源、时间、影响和下一步。

SWE-agent 的重要启示：把工具接口设计好，常常比换一个更大的模型更先带来收益。

工具接口约定：从“能调用”到“可验证”

接口字段	示例
输入	service=checkout; window=10m; scope=prod-cn
输出	points、query_url、observed_at、partial、error_class
副作用	read-only / writes-config / restarts-pod
权限	role=sre-oncall; namespace 白名单
时间与并发	timeout=5s; max_parallel=3
验证	healthy_check、rollback_tool、success_condition

方案设计 把“查询近 10 分钟 checkout P95”改写成一个具备输入、输出、权限和失败语义的工具。

幂等与最小动作：执行层的两条底线

幂等 同一个请求重试一次，不应把副作用扩大两次。例：以变更单 ID 作为幂等键。

最小动作 只改变能够验证假设的最小对象。例：删除故障注入对象，而不是重启整个服务。

- ▶ “最小” 不等于 “永远无害”，仍需说明影响面；
- ▶ 工具层应拒绝缺失幂等键、回滚或成功条件的高风险动作；
- ▶ Agent 负责提出动作，人或策略引擎负责判断是否允许。

上下文快照：为什么故障排查需要“当时的世界”

- ▶ 服务版本、配置摘要、拓扑快照；
- ▶ 指标聚合窗口和采样策略；
- ▶ 日志过滤条件与 TraceID；
- ▶ 当前告警、工单和变更事件。

快照键 `incident_id`、`observed_at`、`scope`、`data_watermark`、`trace_id`

不保存快照，恢复/复盘时就可能把今天的数据误当成事故发生时的证据。

记忆分层：短期状态、长期知识、可复用规则

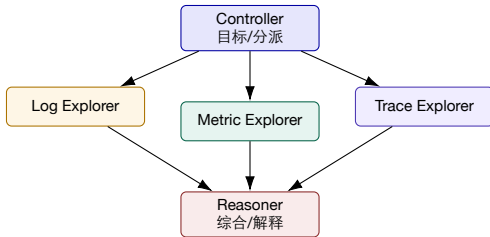
短期记忆 本次事故的观察、假设、工具结果和待办。

长期知识 服务拓扑、SLO、Runbook、历史事故和负责人。

规则记忆 从多次事故总结出的条件—结论—验证—动作。

- ▶ 短期记忆需要严格时间边界；
- ▶ 长期知识需要版本、有效期和适用范围；
- ▶ 规则记忆需要证据支持和人工确认，不能把一次猜测直接写成规则。

多智能体：拆的是能力，不是责任



拆分之后，仍需共享 Trace、证据格式、失败语义和协调器；否则只是多个互相猜测的聊天窗口。

多智能体的协调失败长什么样？

失败模式	表现	设计补丁
重复调查 证据冲突 循环分派 责任漂移	Log、Metric Agent 都查同一时间窗 两个 Agent 对“异常”定义不同 Controller 不断把相同任务发回去 每个 Agent 都说“请另一个确认”	共享任务队列、结果缓存 统一 schema、时间水位和置信度 任务签名、最大轮数、停机条件 明确最终汇总与接管者

判断 多 Agent 的收益来自 “不同视角 + 可合并证据”，不是来自 Agent 数量。

第二部分小结：把 Agent 变成 workflow 组件

1. ReAct 提供“思考—动作—观察”的循环；
2. StateFlow 提供显式状态、守护条件和终态；
3. Tape 提供可恢复、可回放、可评测的语义轨迹；
4. ACI 和工具接口约定决定模型是否能安全使用系统；
5. 多 Agent 必须共享证据、任务状态和责任边界。

三、运维任务工作流

把“日常工作”拆成可重复、可验证的任务单元

任务一：问答必须落到服务和证据

用户问题

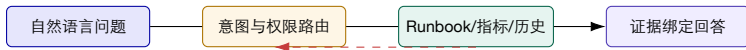
“为什么 checkout 今天比昨天慢？”

workflow展开

1. 解析服务、基线、时间窗；
2. 查询 SLO、流量、版本、依赖；
3. 找出变化最大的候选；
4. 返回事实、假设、链接和未知项。

一个好的回答不是把所有数据贴出来，而是把用户的问题转换成可复核的查询计划。

检索增强：先找对知识，再生成解释



- ▶ 检索结果带版本、适用服务和有效期；
- ▶ 对“没有找到证据”给出明确答复；
- ▶ 高风险问题先做权限和范围检查，再进入检索。

查询翻译：自然语言到观测查询

自然语言槽位	例子	查询约束
对象	checkout / retriever	服务、集群、命名空间必须可解析
指标	p95、错误率、TTFT	指标名称和聚合方式固定
时间	今天、过去 10 分钟	转成绝对时间与时区
对照	昨天同一时段、健康实例	需要明确基线
权限	生产只读	禁止隐式扩展 scope

验证点 如果系统不能把查询计划展示给用户，就不应让模型直接扩大查询范围。

任务二：巡检是“固定清单 + 异常解释”



- ▶ 清单负责覆盖率；
- ▶ 规则负责确定性检查；
- ▶ Agent 负责跨数据源串联、解释和提出下一步；
- ▶ 人负责确认优先级与业务影响。

任务三：告警摘要要回答“所以呢？”

原始告警流

- ▶ 许多高频、重复、局部抖动；
- ▶ 单条告警缺少服务依赖和历史；
- ▶ 值班人需要手工拼接时间线。

可行动摘要

- ▶ 哪些告警属于同一事件？
- ▶ 影响的服务与用户是什么？
- ▶ 哪些证据支持当前判断？
- ▶ 建议观察、升级或处理什么？

摘要的目标是缩短“看到告警”到“做出下一步判断”的时间，不是把告警变得更漂亮。

任务四：事故分诊的输出是“下一步”

分诊问题	示例回答	证据
影响面 是否正在扩大 先看什么 需要谁接管	checkout API，华南集群，约 12% 请求是，过去 5 分钟斜率为正 网络策略与 Endpoint，再看后端进程 平台管理员确认网络动作	区域、调用链、错误率 时间序列与对照 拓扑、readiness、变更 权限与动作风险

调查顺序 “根因是什么”通常不是分诊的第一问；先缩小影响面和证据范围。

证据矩阵：让多个来源相互校验

假设	指标	日志/事件	反证或验证
NetworkChaos Endpoint 为空 后端进程故障 负载过高	跨边界错误升高 readiness 正常但路由失败 readiness、错误率同时恶化 CPU、队列、等待升高	存在故障注入对象 EndpointSlice 无后端 重启/崩溃事件 请求突增	删除对象后恢复 直接访问后端应正常 readiness 稳定则降低概率 限流后曲线回落

矩阵让 Agent 的每个候选都必须提供支持证据和反证路径。

假设驱动诊断：不要只按“最像的答案”排序



- ▶ 先问“哪个查询能区分候选”，再问“哪个候选概率最高”；
- ▶ 证据不增时停止循环；
- ▶ 诊断结束条件：根因足够明确、动作明确、或必须人工接管。

任务五：SOP 执行的关键不是生成命令

生成命令前

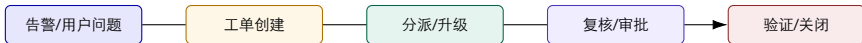
- ▶ 目标对象和版本是否正确？
- ▶ 是否在维护窗口？
- ▶ 影响面和回滚步骤是什么？
- ▶ 是否满足审批策略？

生成命令后

- ▶ dry-run 或小范围灰度；
- ▶ 执行结果结构化记录；
- ▶ 业务与系统双重健康检查；
- ▶ 超时自动停机，失败转人工。

底线 “命令看起来合理” 不是执行许可；动作许可来自策略、权限、审批和验证。

任务六：流程管理让 **Agent** 进入工单和班次



- ▶ Agent 可以自动填充影响面、时间线、证据链接和候选负责人；
- ▶ 工单是跨班次的共享状态，不应只存在于对话窗口；
- ▶ 关闭工单前必须有验证证据和后续行动项。

与 ITSM / ChatOps 集成：消息不是状态机

ChatOps 适合通知、提问、审批和接管；不适合保存完整原始证据。

工单 适合责任、时间线、状态和审计；需要链接到原始查询。

运行平台 适合执行、回滚、健康检查；必须与审批策略绑定。

三个系统应共享 incident_id / change_id，而不是依赖人肉复制粘贴。

企业 workflow 基准：任务要有步骤和验证器

682

WorkArena++ 报告的企业流程任务规模

- ▶ L2：有明确步骤的浏览器任务；
- ▶ L3：工单 + 知识库 + 多步流程；
- ▶ 任务成功需要环境状态验证，不是语言相似度。

基准结果与模型、浏览器、任务子集和验证器绑定；它启发我们如何评测，不是生产 SLA。

验证器：Agent 完成“文本任务”还不够

任务	表面输出	真正验证
生成巡检报告	文本完整、语气自然	指标链接可打开、时间窗正确、异常覆盖
创建工单	返回 ticket_id	工单字段齐全、负责人和优先级合理
执行限流	命令返回成功	目标流量下降、SLO 未恶化、可回滚
关闭事故	写了“resolved”	业务回归、影响恢复、后续项入库

原则 每个可自动化任务必须有 oracle、validator 或可观察的状态变化。

评价一个运维 Agent：五个维度

结果

任务是否解决

证据

是否可复核

安全

是否越权

成本

时延与 Token

采用

人是否愿意接管

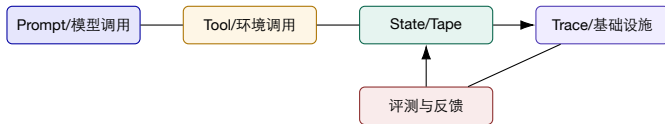
只报“准确率”会掩盖：错误动作、无法解释、成本过高和运维人员拒绝使用。

轨迹指标：把一次任务拆开测

指标	解释
Task success	最终状态是否满足业务/系统验证器
Evidence coverage	结论中有多少关键主张绑定了来源
Tool precision	工具调用是否命中目标、参数是否正确
Time to useful action	从告警到第一个有信息增益的动作
Human takeover rate	何时、为什么需要人接管
Unsafe action rate	越权、无审批、无回滚或扩大影响面的动作比例

轨迹指标要按任务类型和风险分层，不能把低风险问答与生产变更混为一谈。

把 Langfuse / Tape / Trace 放在同一张图里



- ▶ Prompt/模型层：版本、Token、延迟、输入输出；
- ▶ Tool/状态层：参数、结果、异常、转移；
- ▶ 基础设施层：服务、容器、网络、CPU、GPU；
- ▶ 评测层：任务成功、证据、安全和人审反馈。

版本化：没有版本就没有可解释的回归

一起记录

- ▶ 模型与推理参数；
- ▶ Prompt、工具 schema、策略；
- ▶ Runbook、知识库和索引；
- ▶ 系统版本、配置和数据水位。

这样才能回答

- ▶ 为什么昨天能诊断、今天不能？
- ▶ 是模型变了，还是工具变了？
- ▶ 新规则降低了噪声，还是漏掉了告警？
- ▶ 这次动作是否由哪个版本触发？

第三部分小结：从任务单元到流程系统

1. 问答、查询、巡检、RCA、SOP 和工单是不同任务类型；
2. 每种任务都要有输入、状态、证据、验证器和终态；
3. 消息、工单、执行平台共享事件 ID，才能跨系统复盘；
4. 评测同时看结果、证据、安全、成本和采用度。

四、近期系统研究

从多模态 RCA，到告警生命周期和跨层追踪

OpsLens 试图解决什么？

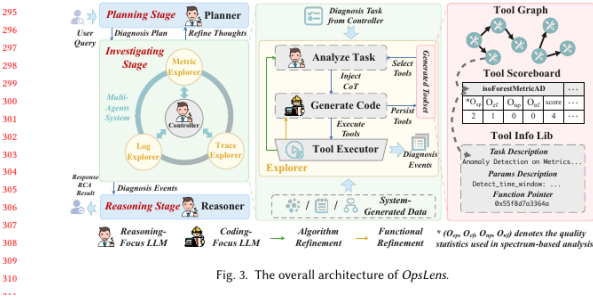
传统 LLM-RCA 的缺口

- ▶ 依赖人工报告或预先写好的工具；
- ▶ 直接把海量原始观测数据塞给模型；
- ▶ Trace、Log、Metric 各自为战；
- ▶ 工具生成后缺少评分、复用和淘汰。

OpsLens 的回答

- ▶ 规划—调查—推理三阶段；
- ▶ Trace / Log / Metric Explorer 专门化；
- ▶ 通过生成工具压缩数据并反馈修正；
- ▶ 用 Tool Graph、Scoreboard、Library 管理工具。

OpsLens 总体架构：规划、调查、推理



阅读方法：先看三阶段如何分工，再看 Explorer 如何生成并管理工具；不要只把它理解成“多 Agent 并行”。

三阶段为什么比一个大 Agent 更容易讲清楚？

Planning 把事故问题变成结构化诊断计划；允许后续根据新证据修订。

Investigating Controller 把任务分派给 Trace、Log、Metric Explorer，收集诊断事件。

Reasoning Reasoner 在调查足够后总结根因和响应，而不是边查边直接下结论。

工程启示 把“搜集证据”和“下结论”分开，可以减少中间假设被过早固化。

工具生成：模型不直接读完所有原始数据



- ▶ 工具把复杂观测数据转换成更小、更有结构的诊断事件；
- ▶ 反馈轮次不是越多越好，要观察完成率、正确性和成本；
- ▶ 生成工具必须受沙箱、权限和资源限制约束。

工具图、评分板、工具库：让工具成为长期资产

Tool Graph 记录工具之间的拓扑和调用顺序。

Tool Scoreboard 记录输入、输出、耗时、成功率和当前质量。

Tool Library 存放描述、版本、适用系统和淘汰标记。

- ▶ 软件系统变化后，工具可能失效；
- ▶ 动态选择工具比固定调用旧工具更适应上下文；
- ▶ 但动态选择必须有白名单、回归测试和人工可见性。

OpsLens 结果怎么读?

0.80

OpenRCA score rate (论文报告)

2.04–4.75×

相对基线的提升范围 (论文报告)

< 5 min

作者报告的生产诊断耗时

- ▶ 这些数字依赖数据集、系统、模型、提示和评测定义；
- ▶ 论文实验与单一公司生产部署不能互相替代；
- ▶ 工具质量管理和多模态协作可用于其他运维场景。

WeRCA: 先构造失败上下文，再让 RCA Agent 推理

问题

- ▶ 原始观测数据碎片化；
- ▶ LLM 直接面对大量细节，难以逐层收敛；
- ▶ 历史 RCA 中的规则没有持续回流。

方法

- ▶ 在线阶段：Triage + 专家 Agent + RCA Agent；
- ▶ 离线阶段：Rule Agent、Domain Expert、Judge Agent；
- ▶ 收集运行中的失败案例，离线修改规则，再用保留的案例检查改动。

WeRCA 总体架构：在线诊断与离线知识演化

RCA framework.

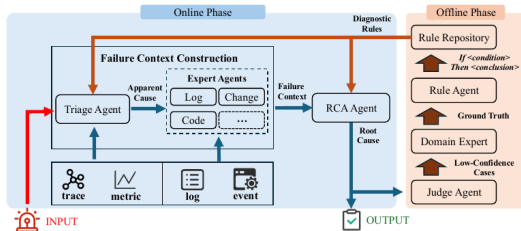
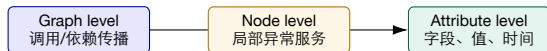


Fig. 1. The overview of WeRCA.

在线负责当下事故；离线负责把高质量、低置信度和人工确认的案例分别处理。

三级失败上下文：从拓扑缩到属性



- ▶ Graph: 先知道异常从哪里传播到哪里;
- ▶ Node: 定位哪个服务或实例的行为不正常;
- ▶ Attribute: 找到足以区分假设的细粒度证据;
- ▶ 逐层缩小搜索空间, 减少把整个系统塞给 LLM。

在线与离线：把“事故响应”和“系统学习”分开

在线阶段

- ▶ 多模态证据构造失败上下文；
- ▶ 输出当前根因和验证建议；
- ▶ 延迟和信息完整性优先。

离线阶段

- ▶ 从历史案例抽取可复用规则；
- ▶ 对低置信度案例请求专家判断；
- ▶ 将确认后的规则带版本回流在线阶段。

事故中的一次错误结论不能直接变成永久规则；离线阶段是风险隔离层。

WeRCA 的结果与局限：数字要带条件

>40 pp

论文报告的部分场景提升

71.43%

WeChat 场景报告结果

2

主要评估系统数量

- ▶ 论文指出完整规则召回、模型波动和系统覆盖仍是限制；
- ▶ 在线诊断必须能在证据不足时停止，而不是强行套规则；
- ▶ 规则仓库本身也要做版本、适用范围和回滚管理。

AlertGuardian: 告警生命周期的三个阶段

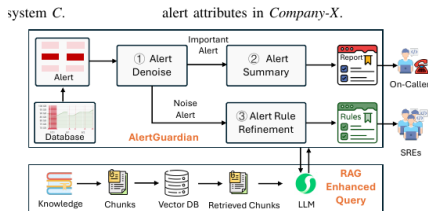
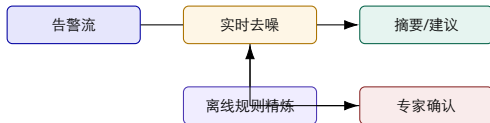


Fig. 9: Overview of AlertGuardian in *Company-X*.

environments, rendering static rules increasingly susceptible to persistent false alarms. Because SREs are often preoccupied

三段式 实时去噪 → 重要告警摘要 → 离线规则精炼；实时和离线承担不同目标。

告警生命周期：实时路径与离线路径



实时路径追求吞吐和及时性；离线路径追求规则质量和可控变更。

告警误报后，谁确认问题、修改规则并复测？

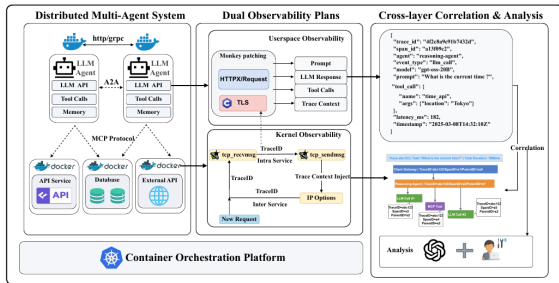
模型反馈 哪些告警被合并、摘要是否引用了正确知识。

工程师反馈 根因是否有用、建议是否可执行、是否漏报。

规则反馈 规则何时生效、误报/漏报、影响哪些服务。

- ▶ 人审不是“给模型打分”这么简单，而是改变未来的规则和知识；
- ▶ 反馈要绑定告警版本、服务和时间段；
- ▶ 规则更新需要回归集和灰度，而不是直接覆盖线上。

AgentTracer: 为什么需要跨语义与基础设施的追踪?



一个用户问题可能跨 Agent、LLM、MCP/HTTP、容器和网络；只在应用层追踪会丢掉跨边界因果线索。

双平面追踪：语义事件 + 内核传播

User-space semantic plane

- ▶ Prompt、LLM response、tool call;
- ▶ Agent、task、parent/child span;
- ▶ 高语义信息，需脱敏与访问控制。

Kernel / infra plane

- ▶ eBPF 观察 TCP、进程和跨服务传播;
- ▶ 轻量 TraceID 通过网络边界传递;
- ▶ 帮助回答“语义步骤卡在哪一层”。

关键判断 追踪不是多存一些日志，而是建立可关联的因果上下文。

AgentTracer 的 Trace / Span 结构

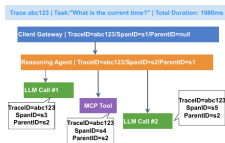


Figure 2: An example trace spanning a 3-service DMAS workflow, illustrating hierarchical span relationships and the causal decomposition of a user query into agent-level execution steps.

- ▶ Trace: 一个用户任务的完整生命周期;
- ▶ Span: 一个 Agent、LLM 或工具执行单元;
- ▶ Parent/Child: 重建分解和并行关系;
- ▶ 与基础设施指标关联: 延迟、CPU、网络 and 错误。

借鉴 **TELLER**：为调查轨迹保存结构化证据

轨迹视角

- ▶ 用户目标、模型计划、工具调用、观察、最终答案；
- ▶ 对齐“哪一步开始偏离”；
- ▶ 区分计划错误、工具错误、环境错误和验证错误。

运维映射

- ▶ 诊断 **Agent** 为什么选择了错误指标？
- ▶ 哪个工具返回空结果却被当成正常？
- ▶ 哪一步把建议越权成了执行？

AgentChaos: 把 Agent 当作需要做可靠性测试的系统

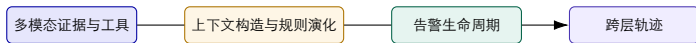
输入扰动 截断、缺失、冲突、延迟或污染观测。

工具扰动 空结果、错误 schema、超时、权限拒绝。

轨迹验证 检查是否循环、越权、幻觉和不安全动作。

- ▶ “正常路径通过” 不等于 workflow 可靠；
- ▶ 要构造负面场景，并预先定义安全终态；
- ▶ 可靠性测试的对象包括模型、工具、状态机和策略的组合。

四项研究放在一条链上

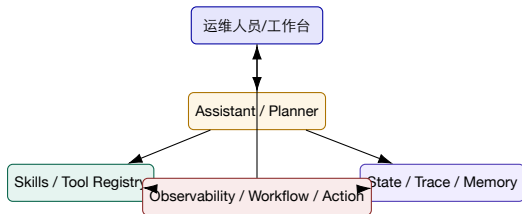


- ▶ OpsLens / WeRCA: 回答“如何调查和理解”;
- ▶ AlertGuardian: 回答“如何管理告警生命周期”;
- ▶ AgentTracer: 回答“如何知道系统实际执行了什么”;
- ▶ 合起来才接近企业级 Agent 的可解释、可持续运营基础。

五、AgenticSRE 与项目探索

从论文机制到面向运维人员的工作台

AgenticSRE：把智能体放进可观测性和工具层



这是根据 AgenticSRE 系统原型重绘的架构示意；界面截图随后作为独立图片展示。

Skills：把可复用运维能力封装起来

Skill 类型	包含内容	例子
观察 Skill	查询模板、过滤、聚合、链接	“服务健康检查”
诊断 Skill	假设空间、证据矩阵、验证步骤	“网络分区 RCA”
流程 Skill	SOP、审批、工单、回滚	“扩容前检查”
治理 Skill	权限、脱敏、审计、成本预算	“生产写操作审批”

- ▶ Skill 不是一段 Prompt，而是“描述 + 工具 + 状态 + 验证器 + 版本”；
- ▶ 技能注册表应支持适用范围、依赖、风险等级和淘汰。

项目工作台：先看系统，再问 Agent



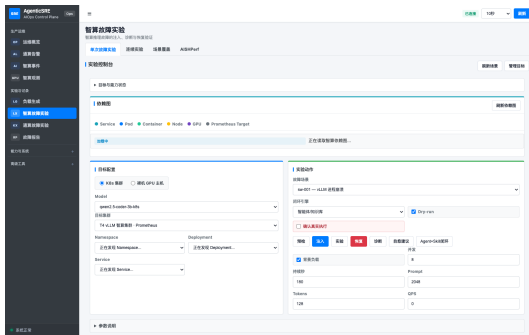
界面中的拓扑、服务、依赖和健康状态，为后续对话提供“当前世界”的上下文。

Assistant: 让人看到“问题—证据—下一步”



对话窗应与证据、工具调用和审批状态并列，而不是只展示一段自然语言。

Workflow: 把一次对话变成可检查的过程



- ▶ 节点是步骤、工具或审批点；
- ▶ 连线表达条件和数据依赖；
- ▶ 运行后应能回到每个节点查看输入、输出和耗时。

Hypothesis 控制：把诊断过程变成可比较的实验

假设对象

- ▶ 候选原因、支持证据、反证；
- ▶ 下一条最有信息增益的查询；
- ▶ 当前信念、冲突和过期时间。

实验约束

- ▶ 只读查询优先；
- ▶ 干预要有影响面和停止条件；
- ▶ 每次实验结果回写到同一事件轨迹。

Hypo / OpsLoop 属于系统原型探索，本页提取其机制，尚不能据此判断其在生产环境中的完整处理效果。

诊断置信度与执行许可必须分开



重要 高置信度 + 高风险，仍需审批；低置信度 + 低风险，可以先做只读验证。

HITL：人不是“最后按一个按钮”

Before 确认范围、目标、风险、回滚和审批人。

During 观察影响面、异常、超时和停止条件。

After 验证系统与业务恢复，确认轨迹和后续行动。

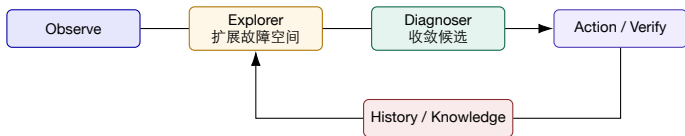
- ▶ 人审界面应展示证据和差异，而不是只显示“允许/拒绝”；
- ▶ 高风险动作允许双人复核或分阶段灰度；
- ▶ 人接管后，Agent 仍应继续记录状态，避免上下文断裂。

ActionStack: 执行、回滚、验证必须成组出现



- ▶ Pre-check 不通过，Action 不得执行；
- ▶ Health check 不通过，进入停止或回滚；
- ▶ Rollback 成功也要验证业务，而不是只看命令返回值。

OpsLoop: 从一次 RCA 到持续演化



OpsLoop、AgenticSRE 属于项目阶段性探索；这些原型用于说明“诊断—知识—再诊断”的演化思想。

证据边界：Demo、论文、生产证据要分开

证据类型	可以支持什么	不能直接支持什么
论文图与实验 项目架构/界面 单个生产案例	方法、实验设置、局限 产品形态、 workflow 概念、当前探索 某类环境中的可行性	所有企业系统都能达到同样效果 长期稳定性和规模化 SLA 通用准确率、跨组织外部有效性

研究生要求 看到一个漂亮的 UI，要追问：状态在哪里？工具边界在哪里？验证器在哪里？失败时谁接管？

六、综合案例与方案设计

用完整工作流回答：现在该查什么、做什么、谁批准？

案例 B：网络分区导致 503——症状与假设

症状

- ▶ frontend-proxy: 200 → 503;
- ▶ backend readiness: 正常;
- ▶ 直连后端正常, 跨边界访问失败。

候选假设

1. NetworkChaos 故障注入;
2. Endpoint / EndpointSlice 为空;
3. backend 进程异常;
4. 流量或资源负载过高。

第一步 只读查询故障对象、Endpoint、跨边界路径和资源指标; 暂不重启、扩容或改路由。

案例 B：证据链与最小修复

步骤	观察	结论/动作
对象检查	NetworkChaos 对象存在	H1 升高，仍需验证
跨边界对照	proxy→backend 失败，backend 直连正常	支持网络路径问题
端点检查	readiness 和 Endpoint 有效	H2、H3 降低
最小干预	删除故障注入对象	只改变故障源，不重启全服务
回归验证	503 恢复，跨边界请求正常	根因得到干预支持

如果删除后没有恢复 停止自动循环；保留轨迹，转平台管理员检查网络策略和节点状态。

案例 C: KV Cache 压力——不要把“慢”直接解释成 GPU 不够

观测

- ▶ Cache: 99%;
- ▶ 内存: 98%;
- ▶ waiting: 12;
- ▶ TTFT: 8.2s; TPOT: 0.8s。

候选

- ▶ KV Cache 被长上下文占满;
- ▶ 计算饱和;
- ▶ 请求队列堆积;
- ▶ 进程异常或内存回收抖动。

先暂停/限流触发负载，观察 waiting、TTFT 和 Cache 是否回落，再决定是否扩容或调整上下文策略。

案例 C：审批卡设计

动作	风险	需要的人与验证
停止压测/限流	中	值班工程师确认；看 waiting 和 TTFT 回落
修改最大上下文	中高	模型服务 Owner；灰度 + 质量回归
扩容 GPU	高	平台负责人；容量、成本和回滚计划
杀进程/重启服务	高	双人审批；确认缓存重建、业务恢复和告警变化

讨论 哪一个动作能最大区分假设、又最不容易扩大影响面？

为“每日巡检数字分身”画一张状态图

1. 选择一个服务：checkout、retriever 或 payment；
2. 写出至少 5 个状态：Init、Observe、Compare、Explain、Approve、End；
3. 为每个状态写一个工具接口约定和一个不变量；
4. 指定一个需要人工确认的节点；
5. 指定一个失败终态和一个回滚/补偿动作。

小组产出不是 Prompt，而是一张可审计的工作流图 + 一页证据账本。

方案评估标准：一张图如何判定“能不能上线试点”

维度	合格表现	常见问题
边界 状态 证据 安全 验证 可运营	服务、角色、权限、动作范围明确 有成功、失败、超时、接管终态 每个结论绑定来源和时间窗 高风险动作有审批、幂等、回滚 有环境与业务验证器 有轨迹、版本和反馈回路	把“运维”当成无限权限 只有一条快乐路径 用模型置信度替代证据 直接执行生成命令 只看命令返回 0 出错后无法复盘

写工具超时：为什么下一步应查状态而不是立即重试？

Agent 提交动作 a17，服务端已接受但响应丢失。客户端只知道“未收到结果”。

步骤	观测	状态更新	允许的下一步
提交	a17 调用超时	UNKNOWN	按动作 ID 查执行账本
查账本	a17=RUNNING	执行中	继续轮询；不重复提交
再次查询	a17=SUCCEEDED	动作完成	检查服务恢复条件
终态验证	错误率仍高	恢复失败	保留结果并升级调查

若查不到记录，也要区分“明确未接受”与“账本不可用”；后者仍是 UNKNOWN。

推导与判断 动作完成与故障恢复是两个状态。需要重试时，只有服务端真正支持幂等语义，复用同一个幂等键才有意义。

算例使用假设数据。

机制依据：AWS Builders’ Library：幂等重试；核对日期：2026-09-09。

参考材料：本讲主要依据

1. OpsLens: Root Cause Analysis on Multi-Modal Observability Data via Multi-Agent Collaboration and Tool Management, 2026;
2. WeRCA: Multi-modal Failure Context Construction and Iterative Enhancement for Root Cause Analysis, 2026;
3. AlertGuardian, ASE 2025; AgentTracer, 2026;
4. Yao et al., ReAct, ICLR 2023; StateFlow, COLM 2024;
5. TapeAgents: A Holistic Framework for Agent Development and Optimization;
6. SWE-agent, ICLR 2024; WorkArena++;
7. AgenticSRE、OpsLoop 系统原型及其独立图片;
8. 课程目录‘课件/’中的企业工作流、Agent 安全、RCA 和可观测性材料。

论文图在本讲用于方法分析；数据结果均保留原论文的实验条件和限制。

下一讲预告：企业级智能运维的转型路径

组织 角色、责任、技能和接管机制。

数据 知识、指标、日志、工单与权限治理。

平台 从试点工作流到可复用、可度量、可运营的平台。

第 9 讲先完成一次任务的查询、处置与结果检查；第 10 讲讨论怎样让很多团队长期用起来。

谢谢

问题、案例与 workflow 设计